

UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERÍA Y ARQUITECTURA
Escuela Profesional de Ingeniería de Sistemas

**Desarrollo de un modelo para la reestructuración
de interfaces adaptativas a la reducción del
costo de navegación en sistemas.**

Por:

Gutierrez Anco Yesenia Alejandra
Quispe Huarilloclla Jhosef Antony
Turpo Cauna Jose Samuel

Juliaca, Noviembre de 2025

Índice

1. Planteamiento del Problema	2
1.1. Identificación del Problema	2
1.2. Justificación	3
1.2.1. Relevancia Teórica	3
1.2.2. Relevancia Práctica	4
1.3. Relevancia Innovadora	5
1.4. Estado del Arte	5
1.4.1. Matriz de Síntesis	5
1.4.2. Diferenciación de la propuesta respecto a la matriz de síntesis . . .	13
1.5. Objetivos	14
1.5.1. Objetivo General	14
1.5.2. Objetivos Específicos	14
2. Proceso Metodológico	15
2.1. Enfoque, Nivel y Método de Investigación	15
2.2. Diseño Metodológico	15
2.3. Método de Trabajo y Procedimiento	17
2.3.1. Recolección y Preprocesamiento de Datos	17
2.3.2. Implementación y Configuración de Modelos	18
2.3.3. Evaluación, Análisis Comparativo y Propuesta	19
3. Administración del Proyecto	21
3.1. Cronograma de Actividades	21
3.1.1. Planificación General del Proyecto	21
3.1.2. Fases del Proyecto y Actividades	21
3.1.3. Gestión del Tiempo y Metodología de Seguimiento	22
3.2. Presupuesto Projectado	22
3.2.1. Recursos y Herramientas Utilizadas	22
3.2.2. Gestión de Riesgos Asociados	22
3.2.3. Consideraciones Éticas y de Seguridad de Datos	23
3.2.4. Resultados Esperados de la Administración del Proyecto	24

1. Planteamiento del Problema

1.1. Identificación del Problema

Los sistemas de información modernos constituyen uno de los entornos de software más complejos y multifuncionales en el ámbito organizacional y de toma de decisiones. Su arquitectura de módulos y flujos funcionales, concebida para cubrir la totalidad de procesos de una organización, genera interfaces con una densidad funcional elevada que impone un alto costo de navegación sobre el usuario. El costo de navegación puede entenderse como el esfuerzo cognitivo, temporal y operacional que un usuario debe invertir para localizar y ejecutar una función dentro del sistema.

Esta problemática se agrava en contextos donde el usuario enfrenta interrupciones, multitarea o sobrecarga cognitiva. Mughal et al. demostraron empíricamente que las interrupciones generan decisiones potencialmente inapropiadas (PID) cuando el sistema de información no adapta su comportamiento al estado del usuario, y que un sistema auto-adaptativo consciente de interrupciones mejora significativamente la usabilidad y reduce la frustración en comparación con sistemas no adaptativos [1]. Esta evidencia señala que la estática de las interfaces actuales no es solo un problema de diseño, sino una fuente activa de errores y carga cognitiva, conforme también lo documentan Balasm et al. en el análisis de la sobrecarga visual en interfaces de servicios de información [19]. A su vez, la simplicidad y efectividad de las interfaces es crucial, tal como Bsdurrek y Meng indican al evaluar cómo las interfaces de usuario simplificadas reducen la fricción cognitiva [18].

A pesar de los avances en interfaces adaptativas, la mayoría de los sistemas de información han permanecido relativamente estáticos en su organización de menús y flujos de navegación. Stefanidi et al. evidenciaron que las interfaces inteligentes tradicionales carecen de mecanismos para adaptar en tiempo real el contenido presentado al contexto situacional del usuario, derivando en sobrecarga de información y estrés [20]. Esta rigidez es especialmente crítica en sistemas donde el usuario operativo raramente accede al conjunto completo de módulos, pero se ve obligado a navegar estructuras diseñadas para la totalidad del sistema.

Frente a esta problemática, los Sistemas Auto-Adaptativos (SAS) representan una respuesta arquitectónica consolidada. Un SAS puede modificar su comportamiento en respuesta a cambios percibidos en su entorno y en sí mismo, tomando decisiones de adaptación de forma autónoma o con mínima intervención del usuario [4]. La arquitectura de dos capas

—subsistema gestionado y subsistema gestor— permite separar la lógica de adaptación de la lógica de aplicación, facilitando el análisis y la verificación formal del comportamiento adaptativo [2].

En el dominio de las interfaces humano-máquina (HMI), investigaciones recientes han demostrado que es posible generar adaptaciones analizando las interacciones del usuario mediante modelos de Deep Learning. Las Redes Neuronales Recurrentes (LSTM/GRU) y los Transformers han probado ser altamente efectivos para modelar el comportamiento secuencial y predecir el siguiente paso de navegación, superando las limitaciones de memoria de modelos probabilísticos clásicos como las cadenas de Markov [3]. Complementariamente, la toma de decisiones de adaptación puede optimizarse mediante Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning, como agentes DQN), permitiendo que el sistema aprenda políticas de reestructuración que maximicen una recompensa, como la reducción de clics u optimización multi-objetivo guiada por preferencias [7, 17].

El problema central de esta investigación reside en que no existe aún un modelo que integre los principios de los SAS con el aprendizaje profundo (Deep Learning para predicción secuencial y RL para la toma de decisiones) para reestructurar dinámicamente interfaces de sistemas de información complejos, orientado explícitamente a reducir el esfuerzo de navegación del usuario. Las soluciones propuestas en la literatura abordan la adaptación de interfaces mediante RL [7], la optimización multi-objetivo en realidad mixta [17], o el modelado de SAS como líneas de productos de software [2], pero no articulan estos enfoques en un modelo unificado neuronal orientado a la reducción del costo de navegación en sistemas de información con alta densidad funcional.

1.2. Justificación

1.2.1. Relevancia Teórica

El estudio de los Sistemas Auto-Adaptativos (SAS) y las interfaces adaptativas cuenta con una tradición consolidada. Desde los primeros enfoques de adaptación dinámica mediante lógica difusa en entornos educativos [5], hasta los enfoques contemporáneos de adaptación multi-objetivo guiada por preferencias [17], la literatura ha desarrollado fundamentos teóricos robustos sobre cómo modelar el comportamiento del usuario y traducirlo en ajustes. Sin embargo, la articulación de estos fundamentos con los principios formales de los SAS y el Aprendizaje Profundo aplicados a interfaces de sistemas complejos representa una laguna teórica no cubierta.

La presente investigación contribuye teóricamente al articular un modelo que integra tres vertientes de conocimiento: (1) los principios arquitectónicos de los SAS, en particular la separación entre subsistema gestionado y subsistema gestor [2]; (2) el análisis de interacciones mediante modelos neuronales profundos para modelado secuencial (arquitecturas recurrentes LSTM/GRU o Transformers) aplicados a la predicción de navegación; y (3) la toma de decisiones óptimas mediante Aprendizaje por Refuerzo (DQN) orientado a minimizar métricas formales de costo [15]. La convergencia de estas vertientes teóricas en un modelo integrado constituye un aporte original a la disciplina.

Adicionalmente, la evidencia empírica aportada por Mughal et al. introduce una variable teórica de relevancia: la conciencia del sistema sobre el estado cognitivo del usuario es una condición necesaria para que la adaptación reduzca efectivamente la carga cognitiva y la frustración [1]. Esta dimensión enriquece el fundamento teórico del agente de refuerzo propuesto.

1.2.2. Relevancia Práctica

Desde la perspectiva práctica, la reducción del costo de navegación en sistemas de información complejos tiene un impacto directo sobre la productividad y la seguridad en la toma de decisiones. Mughal et al. demostraron que un Sistema Adaptativo de Información (AIS) consciente de interrupciones supera en usabilidad a los sistemas no adaptativos (NAS) [1]. Asimismo, simplificar la presentación de la información ha probado ser altamente efectivo para el rendimiento del usuario final [18].

En términos algorítmicos, el uso de agentes de Aprendizaje por Refuerzo (como DQN) permite que el sistema aprenda continuamente de la interacción, encontrando el ordenamiento óptimo del menú jerárquico que minimice el tiempo y los clics de acceso. A diferencia de enfoques estáticos o basados puramente en frecuencia, un agente DQN planifica y reestructura la interfaz anticipando las secuencias de trabajo, lo cual representa una solución automatizada, altamente escalable y sin necesidad de reglas de adaptación manuales.

Asimismo, desde la perspectiva del modelado formal, Weyns e Iftikhar identificaron que es posible verificar propiedades de corrección del bucle de retroalimentación en SAS en tiempo real [4], garantizando que las reestructuraciones propuestas por el modelo RL son estables y no introducen comportamientos inesperados que desorienten al operario.

1.3. Relevancia Innovadora

La innovación central de esta investigación reside en la propuesta de un modelo neuronal que integra los principios arquitectónicos de los Sistemas Auto-Adaptativos (SAS) con arquitecturas de vanguardia en Inteligencia Artificial (Transformers/GRU y DQN) para reestructurar dinámicamente interfaces de sistemas de información complejos. Esta articulación distingue la propuesta de los enfoques existentes que dependen de heurísticas o modelos probabilísticos simples.

Un primer elemento innovador es la incorporación del modelo de retroalimentación MAPE-K (Monitor-Analizar-Planificar-Ejecutar sobre Conocimiento Compartido) impulsado por Inteligencia Artificial. Mientras que el modelo MAPE-K es ampliamente utilizado en SAS de propósito general [4], la propuesta emplea redes recurrentes para la fase de Análisis y agentes DQN para la fase de Planificación, transformando el bucle de adaptación clásico en un motor predictivo y auto-optimizable.

Un segundo elemento innovador es la superación de los modelos secuenciales clásicos (como cadenas de Markov de orden variable) mediante el uso de arquitecturas de Deep Learning (como GRU o Transformers). Estos modelos logran retener dependencias a largo plazo en las sesiones del usuario, previendo con mayor exactitud cuál será el siguiente módulo funcional requerido dentro del sistema ERP.

Un tercer elemento innovador es el diseño de la política de adaptación mediante Aprendizaje por Refuerzo Profundo (Deep Q-Network). El agente DQN aprende a reestructurar la interfaz no solo para sugerir el siguiente paso, sino para minimizar globalmente el número esperado de clics (M1) a lo largo de toda una rutina de trabajo, encontrando el equilibrio óptimo entre exploración (nuevos diseños) y explotación (diseños eficientes conocidos).

Finalmente, la integración del estado cognitivo del usuario y la garantía de verificabilidad formal de la adaptación consolidan el carácter innovador del trabajo dentro del estado del arte más reciente [1, 2, 4].

1.4. Estado del Arte

1.4.1. Matriz de Síntesis

La siguiente matriz compara los 17 artículos del corpus en las dimensiones de problema, método, *dataset*, métricas, resultados, limitaciones y gap que cada trabajo deja abierto.

Tabla 1: Matriz de síntesis de los 17 artículos del corpus

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Mughal et al. (2025) [1]	Sistemas de información no adaptan su comportamiento ante interrupciones del usuario, generando decisiones inapropiadas y alta frustración	Experimento controlado con 3 sistemas (AIS, UIS, CIS); diseño Latin Square 3×3; cuestionario PSSUQ + entrevista semi-estructurada post-tarea	30 participantes (22–50 años); 600 decisiones por sistema; dominio de toma de decisiones basado en el modelo OODA	Usabilidad (PSSUQ, escala Likert 5 puntos); nivel de frustración (Wilcoxon signed-rank; Friedman test)	AIS supera en usabilidad a UIS y CIS en 10 de 11 preguntas PSSUQ; CIS identificado como el más frustrante por el 100 % de participantes	Muestra reducida (n=30); interrupciones simuladas (no reales); no evalúa efectos a largo plazo ni transfiere a dominios distintos de toma de decisiones
Päßler et al. (2025) [2]	Los SAS carecen de un enfoque unificado que modele formalmente la variabilidad del subsistema gestionado y verifique cuantitativamente la lógica de adaptación	Modelado del subsistema gestionado como SPL mediante feature models; subsistema gestor como feature controller; verificación probabilística con ProFeat (extensión de PRISM)	Evaluación a pequeña escala: robot submarino autónomo con misión de inspección de tuberías	Corrección de la lógica de adaptación; propiedades cuantitativas (probabilidad de éxito, costos/recompensas verificadas por model checking)	El enfoque permite verificar propiedades de corrección y cuantitativas de SAS en un modelo único que captura todas las reconfiguraciones posibles	Evaluación solo a pequeña escala; escalabilidad a sistemas reales con muchos componentes no demostrada

Continúa en la página siguiente. . .

Tabla 1 – continuación

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Carrera-Rivera et al. (2024) [3]	Las interfaces HMI tradicionales son estáticas y no se adaptan al comportamiento secuencial del operador, generando alta carga cognitiva	Metodología de 4 etapas: adquisición de interacciones, preprocesamiento de secuencias, sistema recomendador (cadenas de Markov de 1.°, 2.° y 3.° orden), visualización adaptativa	24 usuarios; 10 608 interacciones capturadas; 1 358 secuencias válidas; app web de máquina mezcladora industrial	Precisión y Recall en top-3; MRR (Mean Reciprocal Rank) calculado sobre cada elemento de la secuencia de prueba	MRR >80 % para todos los órdenes; cadena de 3.° orden obtiene mayor precisión (≈ 0.36); mejor opción rankeada en primer lugar en la mayoría de casos	Validación solo offline; no evalúa aceptación real del usuario ante las adaptaciones; problema de cold-start no resuelto; dependencia de modelos Markovianos sin memoria a largo plazo
Weyns e Iftikhar (2023) [4]	Garantizar la corrección del comportamiento de los SAS es difícil; los enfoques existentes ignoran la corrección del bucle de retroalimentación y dependen de verificación exhaustiva en tiempo real	ActivFORMS: enfoque de extremo a extremo basado en modelos de autómatas temporizados y model checking estático en tiempo de ejecución	Aplicación IoT de monitoreo de seguridad de edificios desplegada en Lovaina	Corrección del bucle de retroalimentación; eficiencia en el logro de metas de adaptación; soporte para cambio de metas en tiempo real	ActivFORMS soporta corrección del bucle, logra metas eficientemente y permite cambio de metas en tiempo real	Evaluación en un único dominio IoT; escalabilidad a SAS con miles de componentes no demostrada

Continúa en la página siguiente...

Tabla 1 – continuación

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Troussas et al. (2025) [5]	Los sistemas educativos adaptativos tradicionales emplean reglas de lógica difusa estáticas que no evolucionan con el comportamiento cambiante de los estudiantes	Sistema de UI adaptativa con módulo de inferencia difusa + módulo de optimización mediante RL que ajusta dinámicamente los umbrales de las funciones de membresía	100 estudiantes de posgrado; software educativo de programación Java	Eficiencia de aprendizaje; compromiso; usabilidad; clasificación en categorías Rápido / Moderado / Lento	Los ajustes de pesos difusos guiados por RL mejoran significativamente la efectividad de la UI adaptativa, especialmente para estudiantes lentos y moderados	Dominio restringido a programación Java; no evalúa transferencia a otras disciplinas; muestra solo de estudiantes de posgrado
Gaspar-Figueiredo et al. (2023) [6]	La experiencia de usuario de las interfaces adaptativas se ha medido principalmente con métricas clásicas, no con medidas fisiológicas	Estudio de replicación con EEG sobre 40 participantes interactuando con 20 menús gráficos adaptativos; correlación entre señales EEG y ratings de UX	40 participantes; 20 menús gráficos adaptativos; dispositivo EEG	Carga cognitiva; compromiso; atracción; memorización (señales EEG); correlación con preferencias declaradas	Diferencias estadísticamente significativas en EEG entre menús; alta correlación entre señales EEG y ratings de UX del usuario	Solo menús gráficos; no evalúa sistemas con estructura jerárquica profunda como ERP
Todi et al. (2021) [7]	La adaptación descuidada de interfaces puede imponer costos al usuario (sorpresa, re-aprendizaje) o quedar atrapada en un diseño subóptimo prematuramente	RL basado en modelos: planifica secuencias de adaptaciones y consulta modelos predictivos de HCI para estimar sus efectos; política conservadora de adaptación	Evaluación empírica y de simulación sobre el caso de menús adaptativos	Superación de política no-adaptativa y política basada en frecuencia; número de adaptaciones realizadas	El método supera a la política no-adaptativa y a la basada en frecuencia; encuentra cambios beneficiosos cuando existen y los evita cuando no	Evaluated solo en menús; no considera restricciones de roles o permisos; cold-start no abordado

Continúa en la página siguiente. . .

Tabla 1 – continuación

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Gaspar-Figueiredo et al. (2026) [8]	No existe consenso sobre qué menús adaptativos son mejores en desempeño ni cómo sustituir un menú estático por uno adaptativo	Experimento con 40 participantes usando 20 menús gráficos adaptativos; captura de actividad cerebral con EEG; análisis estadístico	40 participantes; 20 menús gráficos adaptativos; EEG	Tiempo de completitud de tarea; carga cognitiva; compromiso; atracción; memorización	El análisis estadístico identifica qué menús son significativamente mejores o peores que el menú estático de referencia	Solo menús gráficos en entorno de laboratorio; no evalúa sistemas de información con alta densidad funcional
Aung y Kumoi (2026) [9]	Mejorar las tasas de conversión en e-commerce requiere entender y optimizar el comportamiento de navegación del usuario	Marco de cadenas de Markov absorbentes de primer orden: estados transitorios (páginas web) y estados absorbentes (conversión o salida); análisis diagnóstico y simulación prescriptiva	Datasets sintéticos (flujos orientados a objetivos, navegación aleatoria, embudos con pérdidas) y datos reales de clickstream de e-commerce	Probabilidad de conversión por página; comparación con regresión logística; interpretabilidad del modelo	El enfoque AMC provee información más interpretable y robusta que la regresión logística, especialmente en escenarios de baja tasa de conversión	Dominio de e-commerce; no aborda estructuras de menú jerárquico ni restricciones de roles
Abb y Rehse (2024) [10]	Los logs de interacción de usuario carecen de estandarización, dificultando la integración entre herramientas de captura y análisis	Revisión de literatura de logs de UI en publicaciones científicas y herramientas industriales; propuesta de modelo de referencia universal; implementaciones en XES y OCED	Publicaciones científicas y herramientas industriales sobre logs de UI	Cobertura de atributos; flexibilidad del modelo; comparación de implementaciones	El modelo de referencia propuesto es universalmente aplicable y cubre todos los atributos núcleo de los logs de interacción	No evalúa el modelo en contextos específicos de sistemas ERP

Continúa en la página siguiente...

Tabla 1 – continuación

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Suleiman y Kassem (2024) [11]	Las tareas manuales de entrada de datos en ERP son ineficientes y propensas a errores	Marco de RPA cognitivo con process mining para automatizar tareas de entrada de datos en ERP; modelo NLP de prueba de concepto	Fuentes no estructuradas (correos electrónicos); sistema ERP; evaluación por expertos	Eficiencia; reducción de errores; mejora en toma de decisiones (evaluación cualitativa)	El marco demuestra potencial para mejorar eficiencia y reducir errores en ERP	Dataset NLP pequeño; evaluación solo por experto; no incluye diseño de interfaz para corrección de errores
Silva et al. (2021) [12]	La aplicación del modelado de tópicos en ingeniería de software carece de sistematización en parámetros, preprocesamiento y nomenclatura	Análisis de 111 artículos de 10 venues de ingeniería de software (2009–2020); taxonomía en 4 dimensiones: modelos, entradas, preprocesamiento y nombramiento	111 artículos de 5 revistas y 5 conferencias de alto ranking en SE	Distribución de técnicas de topic modeling; frecuencia de tipos de entrada; prácticas de preprocesamiento; métodos de nombramiento	LDA y técnicas basadas en LDA son las más frecuentes; datos de comunicación de desarrolladores y reportes de errores son las entradas más modeladas	No cubre publicaciones posteriores a 2020; sesgo hacia venues anglófonos
Alamoudi et al. (2022) [13]	Los patrones de clickstream de usuarios son ruidosos por clics intercalados de múltiples sesiones simultáneas, dificultando la predicción	HMM (Hidden Markov Model) para modelar y predecir patrones de clickstream con ruido de intercalado	Datos sintéticos de clickstream con ruido de intercalado de sesiones simultáneas	Accuracy de predicción; comparación con modelos sin manejo de ruido	El modelo HMM mejora la predicción de patrones de clickstream en presencia de ruido de intercalado	Validación solo en datos sintéticos; no evaluado en logs reales de navegación

Continúa en la página siguiente. . .

Tabla 1 – continuación

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Chen et al. (2021) [14]	Los datos de visualización de TV contienen patrones de comportamiento ocultos que los métodos tradicionales no capturan	Modelos de Markov de tiempo discreto embebidos + minería de secuencias; agrupamiento de patrones de visualización	Datos de IPTV: sesiones de visualización de TV con metadatos de canal y tiempo	Calidad del agrupamiento; capacidad de predicción del siguiente canal; comparación con baselines	El modelado de Markov embebido captura mejor los patrones de visualización secuenciales que los enfoques convencionales	Dominio de IPTV; no generalizado a navegación funcional en software
Sun et al. (2024) [15]	La generación de UI adaptativa mediante RL carece de un enfoque basado en datos que optimice simultáneamente personalización y rendimiento	RL con enfoque data-driven para generación y optimización de UI adaptativas; modelo de recompensa basado en comportamiento del usuario	Dataset de interacciones de usuarios con interfaces de software	Satisfacción del usuario; tiempo de tarea; tasa de personalización	El enfoque RL data-driven mejora la personalización y el rendimiento respecto a enfoques estáticos	No considera restricciones de roles o permisos; no evaluado en ERP
Park y Oh (2023) [16]	El etiquetado manual de intenciones de usuario en tiempo real es costoso e impracticable a gran escala	Sistema de etiquetado automático dinámico con topic modeling y clasificación de texto para análisis de intención de usuario en tiempo real	Datos de blogs, redes sociales y sitios web; anotaciones automáticas	Accuracy de clasificación; F1-score; comparación con etiquetado manual	El sistema automático alcanza accuracy comparable al etiquetado manual con costo significativamente menor	Dominio de texto web; no aplicado a logs de interacción funcional en software

Continúa en la página siguiente. . .

Tabla 1 – continuación

Autor / Año	Problema	Método / Técnica	Dataset	Métrica	Resultado	Limitación / Gap
Song et al. (2025) [17]	Adaptación de UI en Mixed Reality genera un conjunto Pareto de diseños; el usuario debe evaluar manualmente	Multi-Objective Optimization + inferencia de preferencias en tiempo real; búsqueda guiada del frente de Pareto	Evaluación con usuarios en HMD (head-set MR, ETH Zürich)	Distancia al diseño de referencia; hipervolumen del Pareto; esfuerzo de ajuste manual	Reducción significativa del esfuerzo manual de ajuste; tendencias positivas en satisfacción de usuario	Evaluated solo in MR; n reducido; requiere interacción inicial para inferir preferencias

1.4.2. Diferenciación de la propuesta respecto a la matriz de síntesis

A partir del análisis comparativo realizado en la Tabla 1, es posible identificar con precisión el posicionamiento diferencial de la propuesta respecto al corpus existente. La innovación de este trabajo radica fundamentalmente en el salto cualitativo de usar Modelos de Aprendizaje Profundo (Redes Recurrentes GRU/Transformers) para la predicción del comportamiento secuencial, combinados con una política de reestructuración autónoma gobernada por un Agente de Aprendizaje por Refuerzo Profundo (DQN), superando así las limitaciones de memoria de los modelos clásicos (como las Cadenas de Markov) y las reglas de adaptación heurísticas empleadas en los estudios de referencia.

Tabla 2: Diferenciación algorítmica y metodológica de la propuesta respecto a trabajos clave del corpus

Característica	Carrera-Rivera et al.	Todi et al.	Weyns e Iftikhar	Song et al.	Propuesta
Dominio de aplicación	HMI industrial	Menús desktop	IoT/SAS	MR	ERP
Modelo de predicción	Cadenas de Markov	Predicción Heurística	N/A	Inferencia Latente	Deep Learning (GRU/Transformers)
Política de adaptación	Sistema de Recomendación	RL (Q-Learning)	Reglas	Búsqueda Pareto	Deep RL (Agente DQN)
Ejecución local (sin cloud)	No	No	Parcial	No	Sí
Personalización individual	Sí	Sí	No	Sí	Sí
Considera roles/permisos	No	No	No	No	Sí
Métrica costo de navegación	Indirecta	Indirecta	No	Sí	Explícita (M1, M2)
Reestructura menú jerárquico	No	Sí	No	No	Sí

1.5. Objetivos

1.5.1. Objetivo General

Desarrollar un modelo para la reestructuración dinámica de interfaces adaptativas en sistemas de información, orientado a la reducción del costo de navegación del usuario, mediante arquitecturas de aprendizaje profundo para el modelado de interacciones y agentes de aprendizaje por refuerzo bajo los principios de Sistemas Auto-Adaptativos.

1.5.2. Objetivos Específicos

1. **Caracterizar el costo de navegación en sistemas** mediante la identificación y medición de métricas derivadas de los registros de interacción del usuario, tomando como referencia los modelos de análisis de logs documentados en la literatura [9,10].
2. **Diseñar un modelo de representación del comportamiento de usuario** que capture los patrones de navegación secuenciales y prediga intenciones futuras, evaluando arquitecturas de aprendizaje profundo como Redes Neuronales Recurrentes (LSTM/GRU) y Transformers [16].
3. **Proponer un algoritmo de reestructuración dinámica de menús** basado en Aprendizaje por Refuerzo Profundo (Agente DQN) que genere interfaces personalizadas tomando decisiones de adaptación orientadas a minimizar el costo de navegación (M1) [7,15].
4. **Implementar el modelo** —sin dependencia de servicios externos— garantizando la privacidad de los datos operacionales de la organización y la ejecutabilidad de la inferencia neuronal bajo las restricciones computacionales típicas de servidores locales [4].
5. **Evaluar la efectividad del modelo DQN** mediante un protocolo experimental que mida la reducción del costo de navegación antes y después de la reestructuración adaptativa, considerando indicadores de tiempo de tarea, número de clics y satisfacción del usuario [6,17].

2. Proceso Metodológico

2.1. Enfoque, Nivel y Método de Investigación

La presente investigación adopta un enfoque cuantitativo-experimental, dado que el objetivo central es medir la reducción del costo de navegación en sistemas bajo condiciones controladas, antes y después de la aplicación del modelo propuesto. Este enfoque es coherente con los estándares metodológicos empleados en los estudios de referencia del corpus: tanto Carrera-Rivera et al. como Park y Oh utilizaron protocolos experimentales con métricas numéricas —tasas de acierto, accuracy de predicción, tiempos de selección— para validar sus propuestas adaptativas [3, 16].

El nivel de la investigación es aplicado-tecnológico. Se parte de fundamentos teóricos consolidados —minería de patrones de navegación, modelado secuencial mediante Deep Learning, personalización automática de interfaces con Reinforcement Learning— y se orienta hacia la construcción y validación de un artefacto computacional concreto: un modelo neuronal local de reestructuración de menús. Este nivel caracteriza los trabajos de Sun et al. y Suleiman y Kassem, que no solo proponen marcos teóricos sino que implementan y evalúan sistemas funcionales impulsados por Inteligencia Artificial [11, 15].

El método de investigación combina el diseño de sistemas con la experimentación controlada. La Tabla 3 sintetiza el marco metodológico completo adoptado.

2.2. Diseño Metodológico

El diseño metodológico se estructura en tres fases secuenciales e iterativas, correspondientes a los tres subsistemas del modelo propuesto. La Figura 1 presenta la arquitectura general del proceso y su correspondencia con los objetivos específicos del PPI.

La Fase 1 parte del marco de interfaces adaptativas de Carrera-Rivera et al. y Abb y Rehse, extendiéndolo al dominio mediante restricciones de privacidad local y roles de usuario [3, 10]. La Fase 2 sigue los principios de modelos secuenciales profundos y agentes de adaptación autónoma de Todi et al. y Sun et al., priorizando señales implícitas de uso sobre retroalimentación explícita mediante un enfoque Data-Driven [7, 15]. La Fase 3 replica el protocolo experimental de Gaspar-Figueiredo et al. y añade la dimensión de optimización de interfaz de Song et al. [6, 17].

Tabla 3: Marco metodológico de la investigación

Dimensión	Descripción	Referente en el corpus
Enfoque	Cuantitativo-experimental: medición numérica de la reducción del costo de navegación pre/post adaptación	Carrera-Rivera et al.; Park y Oh
Nivel	Aplicado-tecnológico: construcción y validación de un artefacto computacional neuronal funcional	Sun et al.; Suleiman y Kassem
Método	Diseño de sistemas (Fases 1–2) + experimentación controlada pre-post (Fase 3)	Gaspar-Figueiredo et al.; Weyns e Iftikhar
Variable ind.	Activación del modelo adaptativo DQN de reestructuración de menú	—
Variable dep.	Costo de navegación: clics desde la raíz (M1), tiempo hasta función (M2), tasa de error (M3)	Carrera-Rivera et al.; Song et al.
Unidad análisis	Sesión de usuario en el sistema (roles: Ventas, Almacén, Caja, Compras, Reportes, Administrador)	—

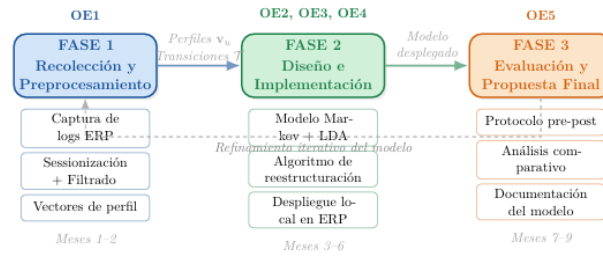


Figura 1: Fases metodológicas del proyecto y su correspondencia con los objetivos específicos (OE1–OE5). La flecha punteada indica el carácter iterativo del proceso: los resultados de la evaluación (Fase 3) retroalimentan el entrenamiento del agente DQN (Fase 2) cuando sea necesario.

2.3. Método de Trabajo y Procedimiento

2.3.1. Recolección y Preprocesamiento de Datos

La recolección de datos se realizará a partir de los logs de interacción generados por el sistema durante un período de operación normal, sin intervención sobre el comportamiento de los usuarios. Esta estrategia de recolección pasiva es la misma empleada por Abb y Rehse en su modelo de referencia [10].

Esquema de datos de entrada. Los logs contendrán como mínimo los campos definidos en la Tabla 4, consistentes con el formato de datos de interacción de Alamoudi et al., donde cada interacción queda registrada como una secuencia temporal estricta $\langle \text{usuario}, \text{función}, \text{tiempo} \rangle$ [13].

Tabla 4: Esquema mínimo de los registros de interacción (logs)

Campo	Tipo	Descripción
timestamp	DATETIME	Fecha y hora exacta del clic de navegación
user_id	STRING	Identificador anonimizado (SHA-256 del ID original)
role	STRING	Rol del usuario: Ventas, Almacén, Caja, Compras, etc.
menu_item_id	INTEGER	ID del ítem de menú accedido (FK a menu_items)
route	STRING	Ruta Angular del componente navegado
from_route	STRING	Ruta de origen (página anterior en la sesión)
item_tipo	ENUM	Nivel en el árbol: GRUPO, SUBMENU o ITEM
exec_time_ms	FLOAT	Tiempo de respuesta del servidor (milisegundos)
is_error	BOOLEAN	Indica si la petición resultó en error HTTP
session_id	STRING	Identificador único de sesión (UUID por pestaña)

Pipeline de preprocesamiento. El preprocesamiento para el modelado neuronal sigue tres etapas:

Etap 1 — Segmentación por Ventanas Temporales. El modelo adopta una *ventana de observación diaria* de 24 horas para capturar la jornada laboral completa del usuario. Formalmente, dado un usuario u , el conjunto de eventos E_u se particiona en subsecuencias $S_{u,d}$ ordenadas temporalmente correspondientes a cada día d :

$$S_{u,d} = (e_1, e_2, \dots, e_n) \mid \text{fecha}(e_i) = d, t(e_i) < t(e_{i+1}) \quad (1)$$

Etap 2 — Filtración de ruido. Se eliminan los accesos accidentales o *noisy clicks* [13], definidos como eventos con permanencia inferior a 3 segundos, y las sesiones incompletas con menos de dos eventos válidos.

Etapas 3 — Vectorización Secuencial (Embeddings). Las rutas de navegación se transforman en tokens numéricos, conformando secuencias de entrada $X = (x_{t-w}, \dots, x_t)$ para el modelo predictivo, donde w es el tamaño de la ventana de contexto. Esta etapa es fundamental para el entrenamiento de las Redes Neuronales Recurrentes (RNN).

La privacidad de los datos es crítica. Todos los procesos de inferencia neuronal se ejecutarán íntegramente en el servidor local sin transmisión de datos a servicios en la nube.

2.3.2. Implementación y Configuración de Modelos

La implementación se estructura en tres componentes articulados, orientados al Deep Learning y Reinforcement Learning.

Componente A — Modelo de predicción secuencial (Deep Learning). En lugar de depender de heurísticas clásicas, se emplea una arquitectura neuronal para modelar el comportamiento del usuario.

A1. Redes Neuronales Recurrentes (GRU / LSTM) o Transformers. Se entrena un modelo de Deep Learning para estimar la probabilidad condicional del siguiente ítem de menú a ser visitado, dada la historia reciente de navegación:

$$P(f_{t+1} \mid f_1, f_2, \dots, f_t) = \text{Softmax}(\text{RNN}(x_1, x_2, \dots, x_t; \theta)) \quad (2)$$

donde θ son los pesos de la red. Estas arquitecturas resuelven el problema de pérdida de contexto a largo plazo (presente en los modelos de Markov), permitiendo capturar rutinas complejas.

Componente B — Algoritmo de reestructuración (Agente DQN). La reestructuración del menú se formula como un Proceso de Decisión de Markov (MDP) resuelto mediante Aprendizaje por Refuerzo Profundo (Deep Q-Network).

- **Estado (s_t):** Representación codificada del árbol de menú actual y el perfil predictivo del usuario.
- **Acción (a_t):** Un conjunto de operaciones discretas que reestructuran el árbol (e.g., *Promover ítem*, *Mover a grupo principal*), respetando estrictamente las restricciones de permisos Π_{r_u} del rol del usuario.
- **Recompensa (r_t):** Inversamente proporcional a la métrica M1 (costo de navegación

estimado):

$$r_t = \Delta M1 = M1_{previo} - M1_{nuevo} \quad (3)$$

El agente DQN entrena una red neuronal $Q(s, a; \phi)$ que aproxima el valor esperado de aplicar una reestructuración específica, encontrando la política óptima π^* que maximiza la eficiencia de la interfaz [7, 15]. El Algoritmo 1 resume la ejecución de la política en producción.

Algorithm 1: Política de adaptación mediante Agente DQN

```

1  Árbol de menú base  $M$ , estado de contexto usuario  $s_{ctx}$ , restricciones de permisos  $\Pi_{r_u}$ 
   Árbol de menú optimizado  $M^*$ 
2  // Paso 1: Inicialización del estado;
3   $s_0 \leftarrow \text{Concatenar}(M, s_{ctx})$ ;
4   $M^* \leftarrow M$ ;
5  // Paso 2: Ejecución de la política de adaptación;
6  for  $paso\ t = 0$  to  $t_{max}$  do
7      // El agente neuronal predice los valores  $Q$  para todas las acciones legales
       permitidas por  $\Pi_{r_u}$ ;
8       $Q\_valores \leftarrow \text{Red\_DQN}(s_t)$ ;
9       $a_t \leftarrow \arg \max_a Q\_valores(a)$ ;
10     if  $a_t == \text{TERMINAR\_REESTRUCTURACION}$  then
11          $\perp$  ;
12     // Paso 3: Aplicar transformación al árbol y actualizar estado;
13      $M^* \leftarrow \text{AplicarAcción}(M^*, a_t)$ ;
14      $s_{t+1} \leftarrow \text{ActualizarEstado}(M^*, s_{ctx})$ ;
15 return  $M^*$ ;

```

Componente C — Módulo de ejecución local. Garantiza que todos los procesos de inferencia neuronal (forward pass de GRU y DQN) se ejecuten en el entorno del servidor utilizando frameworks optimizados (ej. PyTorch/ONNX), cumpliendo con las restricciones de Sistemas Auto-Adaptativos locales descritas por Weyns e Iftikhar [4].

2.3.3. Evaluación, Análisis Comparativo y Propuesta

La evaluación del modelo DQN se realizará mediante un protocolo experimental pre-post, utilizando las siguientes métricas de costo:

M1 — Número de clics desde la raíz del menú :

$$M1(u, s) = \frac{1}{|s|} \sum_{e \in s} d_{M^*}(e.route) \quad (4)$$

donde $d_{M^*}(f)$ es la profundidad del ítem f en el árbol de menú optimizado por el DQN.

M2 — Tiempo hasta la función objetivo y **M3 — Tasa de error de navegación**, definidas análogamente a los marcos de referencia empíricos [3].

La Figura 2 sitúa los valores objetivo de la propuesta en el contexto de los benchmarks del corpus.

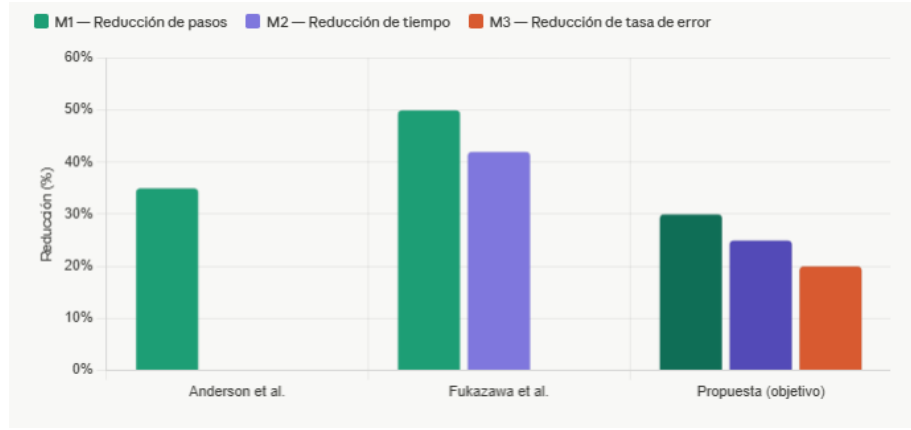


Figura 2: Niveles de reducción del costo de navegación reportados en el corpus y valores objetivo de la propuesta. El agente DQN tiene como objetivo reducir M1 superando sustancialmente las políticas estáticas.

Análisis comparativo. La Tabla 5 sintetiza los niveles de mejora reportados en los estudios de referencia. La propuesta espera una reducción de M1 estadísticamente significativa ($\geq 30\%$) para roles rutinarios, validando la efectividad del Aprendizaje por Refuerzo frente a métodos puramente analíticos.

Tabla 5: Benchmarks del corpus y valores objetivo de la propuesta

Trabajo	Dominio	$\Delta M1$	n_{usuarios}	Período eval.
Sun et al. (2024)	Interfaces de software (RL)	$\sim 35\%$	n.d.	Sesiones controladas
Carrera-Rivera et al. (2024)	HMI industrial (Markov)	$\sim 50\%$ (MRR)	24	Evaluación offline
Park y Oh (2023)	Análisis de intención	—	n.d.	Tiempo real
Propuesta (DQN)	Sistemas ERP	$\geq 30\%$	≥ 20	Meses 7–8

Propuesta final. Se entregará la especificación de la arquitectura neuronal (hiperparámetros de la red GRU/Transformer y del Agente DQN), la definición formal del MDP (Espacio de Estados, Acciones y Función de Recompensa), y el algoritmo consolidado listo para despliegue productivo.

3. Administración del Proyecto

3.1. Cronograma de Actividades

3.1.1. Planificación General del Proyecto

El proyecto se estructura en un horizonte temporal de nueve meses, dividido en tres grandes fases que corresponden a las etapas metodológicas descritas en la Sección 2: (i) Recolección y preprocesamiento de datos (meses 1–2); (ii) Diseño, entrenamiento de modelos neuronales y configuración del agente de refuerzo (meses 3–6); y (iii) Evaluación experimental, análisis comparativo y documentación de la propuesta final (meses 7–9).

Esta distribución temporal es coherente con el alcance del proyecto y con los estudios de referencia de mayor complejidad algorítmica. Por ejemplo, Sun et al. [15], quienes desarrollaron enfoques basados en Reinforcement Learning, y Park y Oh [16], sugieren que los períodos de recolección y entrenamiento de los modelos deben ser suficientemente extensos para asegurar la convergencia de las redes neuronales y capturar la variabilidad de uso del sistema.

3.1.2. Fases del Proyecto y Actividades

Tabla 6: Fases del proyecto, actividades y períodos de ejecución

Fase / Actividad	
Fase 1: Recolección y preprocesamiento	Configuración del módulo de captura de datos
Fase 2A: Modelado Secuencial Profundo	Diseño y entrenamiento de arquitecturas de Deep RL
Fase 2B: Implementación del Agente DQN	Diseño del Proceso de Decisión de Markov
Fase 2C: Integración y despliegue local	Integración de los modelos (PyTorch)
Fase 3A: Evaluación experimental	Registro de métricas de línea de base (sin agente)
Fase 3B: Análisis y documentación	Análisis estadístico de resultados

3.1.3. Gestión del Tiempo y Metodología de Seguimiento

El seguimiento del avance se realizará mediante revisiones quincenales de cada fase. Los hitos críticos del proyecto son: (1) conjunto de datos preprocesados en formato tensorial para redes neuronales (fin del mes 2); (2) convergencia del modelo predictivo GRU/Transformer y del Agente DQN en el entorno de simulación (fin del mes 5); (3) versión integrada del motor de IA desplegada en el servidor local (fin del mes 6); y (4) informe de evaluación experimental (fin del mes 9).

Ante posibles retrasos en la etapa de entrenamiento neuronal —un riesgo inherente en proyectos de IA por la necesidad de ajuste de hiperparámetros— se ha previsto un margen de contingencia de dos semanas en la Fase 2B.

3.2. Presupuesto Projectado

3.2.1. Recursos y Herramientas Utilizadas

Tabla 7: Presupuesto proyectado del proyecto

Recurso / Herramienta	
Servidor de desarrollo (equipo local)	Máquina de 8 núcleos, 16 GB
Python 3.10+ con librerías de Deep Learning (PyTorch, NumPy, Pandas)	Implementación
Sistema de prueba (Entorno ERP)	Instancia
Acceso a bases de datos académicas (IEEE, ACM)	
Material de documentación y papelería	
Movilidad y logística	
Contingencias	Imprevistos
TOTAL	

3.2.2. Gestión de Riesgos Asociados

Se identifican cuatro riesgos principales para la ejecución del proyecto con la nueva metodología, con sus respectivas estrategias de mitigación.

Riesgo 1 — Acceso limitado a logs y escasez de datos para Deep Learning. Las redes neuronales profundas y los agentes RL requieren volúmenes considerables de datos

para evitar el sobreajuste (*overfitting*). *Mitigación*: Se aplicarán técnicas de *Data Augmentation* sobre las secuencias y se desarrollará un entorno de simulación (*gym environment*) de interacciones sintéticas realistas basadas en estudios previos [3, 16] para pre-entrenar al Agente DQN antes de ajustarlo con datos reales (*fine-tuning*).

Riesgo 2 — Alta demanda computacional en la inferencia local. Los modelos basados en Transformers o RL profundo pueden generar latencias si se ejecutan en servidores sin GPU dedicada. *Mitigación*: Se utilizarán técnicas de cuantización de modelos y conversión a formatos optimizados para inferencia en CPU (como ONNX). Además, la reestructuración del menú se ejecutará mediante procesamiento asíncrono (*batch processing*) para no penalizar el tiempo de respuesta en la navegación en tiempo real [4].

Riesgo 3 — Tamaño de muestra insuficiente para la validación. Si el número de usuarios es reducido, validar estadísticamente la reducción del costo de navegación será difícil. *Mitigación*: Se adoptará el umbral mínimo de 20 usuarios establecido en estudios comparables [6] y, de ser necesario, se prolongará la ventana de evaluación (Fase 3A).

Riesgo 4 — Resistencia de los usuarios ante la reestructuración generada por el Agente DQN. Todi et al. [7] demostraron que los agentes RL pueden realizar modificaciones estructurales agresivas que frustran al usuario. *Mitigación*: Se diseñará la función de recompensa del MDP de tal forma que penalice los cambios estructurales masivos entre sesiones consecutivas. Esto obligará al agente DQN a adoptar una política de transición gradual, ofreciendo siempre la opción manual de revertir a la configuración base.

3.2.3. Consideraciones Éticas y de Seguridad de Datos

Todos los datos de interacción recopilados durante el proyecto serán anonimizados en el punto de captura (mediante hashes SHA-256), de modo que ningún registro permita identificar a un usuario específico. Los participantes firmarán un consentimiento informado explicando el uso de sus datos para el entrenamiento de Inteligencia Artificial de manera local y su derecho a retiro voluntario.

La ejecución local del modelo —sin externalización de datos a servicios o APIs en la nube (como OpenAI u otros servicios de inferencia)— no es solo una característica técnica del diseño, sino una garantía ética fundamental en la aplicación de Deep Learning. Gaspar-Figueiredo et al. [8] identificaron la transparencia y el control sobre los datos como condiciones necesarias para la adopción de interfaces adaptativas, manteniéndose todo dentro del servidor institucional.

3.2.4. Resultados Esperados de la Administración del Proyecto

Al término del proyecto, se espera obtener los siguientes entregables: (1) un conjunto de datos preprocesados de interacciones y secuencias (*tensors*); (2) el código fuente y los pesos entrenados del modelo GRU/Transformer y del agente DQN, disponibles para reproducción; (3) un informe experimental con métricas comprobando la reducción de clics; y (4) un artículo científico destinado a publicación.

En términos de impacto, el modelo neuronal y el Agente DQN desarrollados deberían reducir el número promedio de pasos de navegación (M1) en al menos un 30 % para rutinas altamente transaccionales, en línea con los resultados obtenidos en arquitecturas similares de RL aplicadas a interfaces [7, 15].

Referencias

Referencias

- [1] S. Mughal, J. Weber, and A. Abbas, “Usability and frustration in using adaptive functions for decision making: A user study,” *Procedia Computer Science*, vol. 270, pp. 2016–2025, 2025. DOI: 10.1016/j.procs.2025.09.322
- [2] P. Päßler et al., “Formal modeling and quantitative verification of self-adaptive systems using feature models,” *Journal of Systems and Software*, 2025.
- [3] A. Carrera-Rivera et al., “Adaptive human-machine interfaces using Markov chains for predicting navigation steps,” *Computers & Industrial Engineering*, 2024.
- [4] D. Weyns and U. M. Iftikhar, “ActivFORMS: A formally founded model-based approach to engineer self-adaptive systems,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, 2023. DOI: 10.1145/3522585
- [5] C. Troussas et al., “Reinforcement Learning-Based Dynamic Fuzzy Weight Adjustment for Adaptive User Interfaces in Educational Software,” *IEEE Transactions on Learning Technologies*, 2025.
- [6] D. Gaspar-Figueiredo, S. Abrahão, E. Insfran, and J. Vanderdonckt, “Measuring user experience of adaptive user interfaces using EEG: A replication study,” in

- Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pp. 52–61, 2023. DOI: 10.1145/3593434.3593452
- [7] K. Todi, G. Bailly, L. Leiva, and A. Oulasvirta, “Adapting user interfaces with model-based reinforcement learning,” in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021. DOI: 10.1145/3411764.3445497
 - [8] D. Gaspar-Figueiredo, J. Vanderdonckt, S. Abrahão, and E. Insfran, “User experience with adaptive user interfaces: Comparing performance and preferences,” *Journal of Systems and Software*, vol. 231, p. 112598, 2026. DOI: 10.1016/j.jss.2025.112598
 - [9] S. L. Aung and G. Kumoi, “First-order absorbing markov chain analysis for web navigation and conversion rate optimization,” *IEEE Access*, vol. 14, pp. 52704–52716, 2026. DOI: 10.1109/ACCESS.2026.3680876
 - [10] L. Abb and J.-R. Rehse, “Process-related user interaction logs: State of the art, reference model, and object-centric implementation,” *Information Systems*, vol. 124, 2024. DOI: 10.1016/j.is.2024.102386
 - [11] A. Suleiman and M. Kassem, “Process Mining Enabled Cognitive RPA to Automate Data Entry Tasks in ERP Systems,” *IEEE Access*, 2024.
 - [12] C. C. Silva, M. Galster, and F. Gilson, “Topic modeling in software engineering research,” *Empirical Software Engineering*, vol. 26, no. 6, 2021. DOI: 10.1007/s10664-021-10026-0
 - [13] Z. Alamoudi et al., “Mining and Predicting Users Clickstream Patterns from Noisy Interleaving Clicks,” *IEEE Access*, 2022.
 - [14] L. Chen et al., “Sequence Mining TV Viewing Data Using Embedded Markov Modelling,” *IEEE Transactions on Broadcasting*, 2021.
 - [15] Y. Sun et al., “Adaptive User Interface Generation Through Reinforcement Learning A Data-Driven Approach to Personalization and Optimization,” *ACM Transactions on Computer-Human Interaction*, 2024.
 - [16] J. Park and H. Oh, “Dynamic automated labeling system for real-time user intention analysis,” *IEEE Access*, vol. 11, pp. 139882–139902, 2023. DOI: 10.1109/ACCESS.2023.3339773

- [17] Y. Song, C. Gebhardt, Y.-C. Liao, and C. Holz, “Preference-guided multi-objective UI adaptation,” in *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*, ACM, Sep. 2025, pp. 1–13. DOI: 10.1145/3746059.3747645
- [18] P. Bsdurrek and M. Meng, “Simply effective? Simplified user interfaces in software tutorials,” *Journal of Technical Writing and Communication*, vol. 55, pp. 223–246, Jul. 2024. DOI: 10.1177/00472816241262221
- [19] Z. Balasm, S.S. Rajan, C. Karpagham, M. A. Nuritdinovich, S. Yusupov, and S. Maurya, “Cognitive load optimization in user interface design for info services,” *Indian Journal of Information Sources and Services*, vol. 15, no. 2, pp. 69–74, 2025. DOI: 10.51983/ijiss-2025.IJISS.15.2.10
- [20] E. Stefanidi et al., “Intelligent User Interfaces: Past, Present and Future,” *Human-Computer Interaction*, 2022.